

Sprawozdanie z zadania 6

Rozszerzenie gry wyścigowej

Mikołaj Kuryło s193743
Antonina Włodarczyk s197793

1 Sformułowanie zadania i przyjęte założenia

Celem projektu było rozbudowanie świata gry poprzez dodanie możliwości zmiany generowanych map. Mapy te umożliwiają ciekawszą rozgrywkę i dzięki uwzględnieniu

2 Opis modelu map

Mapy zostają dodane w programie dzięki plikom o rozszerzeniu json. Jest to możliwe, dzięki pliku `map_reader`, który służy jako translator.

2.1 Przykładowy kod jednej z map

```
{
  "name": "map_3",
  "width": 5,
  "height": 9,
  "tile_friction": {
    "_": 0.05,
    "X": 0.9
  },
  "start_position": [4, 1],
  "end_position": [4, 8],
  "map": [
    [" ", "X", "X", "X", "X"],
    ["X", "X", " ", "X", "X"],
    ["X", "X", " ", "_", "_"],
    ["X", "X", "X", "X", "_"],
    ["_", "_", "_", "X", "_"],
    ["_", "X", "X", "X", "_"],
    ["_", "X", "_", "_", "_"],
    ["_", "X", "X", "X", "X"],
    ["_", "_", "_", "_", "_"]
  ]
}
```

Mapy z tej postaci są czytane głównie dzięki kodu odpowiedzialnemu za konwersję.

```
class MapReader():
    def read_map(self, file_path: str,
                 file_type: MapFileType) -> MapData:

        match file_type:
            case MapFileType.JSON:
                return self._read_json_map(file_path)
```

```

        case _:
            raise ValueError(f"Unsupported map file type: {file_type}")

def _read_json_map(self, file_path: str) -> MapData:
    try:
        with open(file_path, 'r') as f:
            data = json.load(f)

            # Store symbolic map as-is
            symbolic_map_matrix = np.matrix(data["map"], dtype=object)

            # Create traction map by replacing symbols with friction coefficients
            tile_friction = data["tile_friction"]
            traction_grid = [
                [tile_friction[symbol] for symbol in row]
                for row in data["map"]
            ]
            traction_map_matrix = np.matrix(traction_grid, dtype=float)

            # Extract start and end positions
            start_position = Coords(
                data["start_position"][0], data["start_position"][1])
            end_position = Coords(
                data["end_position"][0], data["end_position"][1])

            return MapData(
                symbolic_map_matrix=symbolic_map_matrix,
                traction_map_matrix=traction_map_matrix,
                start_position=start_position,
                end_position=end_position,
                size_x=int(len(data["map"][0]) * const.MAP_SCALE_FACTOR),
                size_y=int(len(data["map"]) * const.MAP_SCALE_FACTOR)
            )

    except FileNotFoundError:
        print(f"Error: Map file '{file_path}' not found.")
        raise
    except KeyError as e:
        print(f"Error: Missing key in map file: {e}")
        raise
    except json.JSONDecodeError:
        print(f"Error: Map file '{file_path}' is not valid JSON.")
        raise
    except Exception as e:
        print(
            f"Error: Unexpected error reading map file '{file_path}': {e}")
        raise

```

3 Dyskusja osiągniętych wyników

3.1 Zalety

Wykorzystywanie pliku `map_reader` znacznie ułatwia i upraszcza dodawanie rozbudowanych oraz znacznie dłuższych tras. Jest to prosty proces, który nie wymaga wielu modyfikacji.

3.2 Ograniczenia i potencjalne ulepszenia

Mapy nadal pozostają nieruchome, oraz nie zawsze jest poprowadzona droga, która wskazuje najszybszą trasę. W niektórych przypadkach może to wprowadzać gracza w zdziwienie, ponieważ cel nie zawsze jest określony. W przypadku gdzie trasa znajduje się w centrum planszy, a na zewnątrz znajdują się elementy tła, to cel może być

gdziekolwiek. Nie ułatwia to gry, a wyjechanie w nieodpowiednim miejscu może skutkować zakończeniem gry z powodu wypadnięcia za mapę.